

Erweiterung des Spiels um einen Factory-Game-Bereich

Prozedurale Kartengenerierung, Bau-, Abbau- und Crafting-System
innerhalb des bestehenden Unity-Projekts

Dieses Dokument beschreibt die Erweiterung des Casino-Simulationsspiels um einen eigenen Fabrik-/Aufbau-Bereich. Ziel war es, neben dem bestehenden Casino eine zweite, prozedural generierte Spielwelt zu schaffen, in der Ressourcen gesammelt, verarbeitet und Maschinen gebaut werden können - inspiriert von Spielen wie *Satisfactory*.

Engine: Unity 6 (6000.3.9f1) · Sprache: C# · Render Pipeline: Universal Render Pipeline (URP)

Inhalt

1. Projektüberblick
2. Prozedurale Kartengenerierung
3. Vegetation, Felsen und Erzvorkommen
4. Wasser: Seen und Flüsse
5. Integration mit dem bestehenden Spieler
6. Baumodus
7. Ressourcen-Abbau
8. Crafting-System und Hotbar-Anbindung
9. Technische Umsetzung und Werkzeuge
10. Wichtige Code-Stellen erklärt
11. Steuerungsübersicht
12. Dateiübersicht

1. Projektüberblick

Das bestehende Projekt ist eine Unity-basierte Casino-Simulation mit einem First-Person-Charakter, Kamerasteuerung und einem Verkaufs-/Hotbar-System. Die Aufgabe war, dieses Spiel um einen eigenständigen Factory-Bereich zu erweitern, ohne das bestehende Casino zu verändern oder zu ersetzen.

Kernidee

- Eine zweite, weit vom Casino entfernte Kartenregion wird prozedural generiert (Terrain, Biome, Wasser, Vegetation, Rohstoffe).
- Die Karte wird einmalig fest in die bestehende Spielszene eingebacken - sie muss beim Spielstart nicht neu berechnet werden.
- Der Spieler kann direkt beim Drücken von Play auf dieser Karte starten und jederzeit per Tastendruck zwischen Casino und Factory-Bereich wechseln.
- Auf der Karte kann man Bäume und Erzvorkommen abbauen, Materialien zu neuen Gegenständen verarbeiten (Crafting) und Maschinen auf einem Bauraster platzieren.
- Alle gesammelten und hergestellten Gegenstände landen in der bereits vorhandenen Hotbar des Casino-Spiels - es gibt nur ein Inventar, keine zwei getrennten Systeme.

2. Prozedurale Kartengenerierung

Die Karte wird nicht von Hand gebaut, sondern von einem Script (**MapGenerator.cs**) zur Entwurfszeit berechnet. Das Verfahren orientiert sich an klassischen Techniken der prozeduralen Geländegenerierung, wie sie in vielen Aufbau- und Survival-Spielen verwendet werden.

2.1 Höhenkarte (Terrain)

Die Höhe jedes Geländepunkts wird über mehrschichtiges Perlin-Rauschen (*Fractal/Layered Noise*) berechnet: mehrere Rauschebenen unterschiedlicher Frequenz und Stärke werden addiert, wodurch sowohl großräumige Hügelzüge als auch feinere Unebenheiten entstehen. Eine zusätzliche **Domain-Warp**-Technik verzerrt die Abtastkoordinaten leicht, bevor die Höhe berechnet wird - das vermeidet die typischen runden "Perlin-Krater" und erzeugt stattdessen organisch wirkende, unregelmäßige Hügelketten. Eine Kennlinie (*Animation Curve*) sorgt zusätzlich für längere, sanfte Ebenen im unteren und mittleren Höhenbereich - ideal zum späteren Bauen.

Nach der Berechnung durchläuft die komplette Höhenkarte einen Glättungs-Pass (3×3-Box-Blur), der scharfe Mini-Kanten zwischen benachbarten Geländepunkten entfernt - solche Kanten würden sonst dazu führen, dass der Spieler beim Laufen an unsichtbaren Stufen hängen bleibt.

2.2 Biome und Bodentexturen

Abhängig von Höhe, Hangneigung und einer zusätzlichen Feuchtigkeits-Rauschebene wird jeder Punkt einer von fünf Bodenarten zugeordnet: Gras, Waldboden, Fels, Sand (Ufer) und Schnee (Berggipfel). Jede Bodenart hat eine eigene, prozedural erzeugte Textur, die zwischen zwei Farbtönen über Rauschen blendet und zusätzlich seltene Akzentfarben einstreut (z. B. gelbe Wildblumen im Gras, Moosflecken auf Fels, Herbstlaub am Waldboden) - das verhindert den Eindruck

einer einfarbigen Kunststoff-Oberfläche.

Eine weitere großflächige Rauschmaske erzeugt gezielt ebene Flächen ("Plateaus"), indem an diesen Stellen die feinen Detail-Anteile der Höhenberechnung ausgeblendet werden. Diese Flächen sind bewusst für den späteren Fabrikbau vorgesehen.

3. Vegetation, Felsen und Erzvorkommen

Bäume, Felsbrocken und Erzvorkommen werden nach dem gleichen Prinzip verteilt: ein Gitter aus möglichen Positionen wird abgetastet, an jeder Position entscheidet eine eigene Cluster-Rauschebene, ob dort etwas entsteht. Dadurch bilden sich natürlich wirkende Gruppen (z. B. Wälder, Steinfelder, Erzadern) statt einer gleichmäßigen, künstlichen Verteilung.

3.1 Bäume

Jeder Baum besteht aus einem verjüngten Stamm (zwei Zylinder-Segmente) und drei nach oben kleiner werdenden, abgeflachten Kronen-"Etagen" - das ergibt aus der Distanz eine erkennbare Nadelbaum-Silhouette statt einer einzelnen Kugel. Gelegentlich (~12 %) wird ein Baum in warmen Herbstfarben statt Grün eingefärbt, für optische Abwechslung.

3.2 Felsbrocken

Felsen bestehen aus zwei bis drei überlappenden, unregelmäßig rotierten und skalierten Würfeln - das wirkt kantiger und natürlicher als eine einzelne Kugel. Sie kommen bevorzugt in Bergregionen vor, vereinzelt auch im Flachland.

3.3 Erzvorkommen

Sechs Rohstoffarten sind vorgesehen: Eisen, Kupfer, Kohle, Stein, Öl und Uran. Jede Art hat eine eigene Cluster-Rauschebene mit eigenem Offset, wodurch unabhängige, natürlich wirkende Adern entstehen. Höhenregeln sorgen für sinnvolle Verteilung - Uran nur in Bergregionen, Öl nur in Gewässernähe, die übrigen Rohstoffe im Flach- und Hügelland. Optisch besteht ein Erzvorkommen aus einem grauen Felssockel mit farbigen Erz-/Kristallklumpen obenauf statt eines einzelnen schwebenden Farbwürfels; Öl wird als dunkle Gesteinsplatten mit einer glänzenden Lache dargestellt.

Wichtiger technischer Punkt: Da diese Objekte aus mehreren kleinen, einzeln versetzten Teilen bestehen, gab es anfangs Lücken in der Kollisionshülle - ein Fadenkreuz-Strahl (z. B. für die Abbau-Interaktion) konnte zwischen den Teilen hindurchgehen, ohne etwas zu treffen. Als Lösung bekommt jedes dieser Objekte zusätzlich einen einzelnen, unsichtbaren Collider, der die gesamte sichtbare Form umschließt und garantiert immer getroffen wird.

4. Wasser: Seen und Flüsse

Statt einer einzigen großen Wasserebene, die das gesamte Tiefland flutet, entstehen echte, unregelmäßig geformte Gewässer:

- **Seen** werden über eine großflächige, weiche Rauschmaske erzeugt: überschreitet der Wert einen Schwellwert, wird das Gelände an dieser Stelle unter das Wasserniveau abgesenkt - es entsteht ein echtes Seebecken statt einer aufgesetzten Fläche.

- **Flüsse** entstehen über eine sogenannte Ridge-Noise-Funktion mit zusätzlicher Verzerrung (Domain Warp): das erzeugt wenige, aber durchgehende, gewundene Linien statt eines dichten, unnatürlichen Geflechts.
- Beide Gewässerarten haben breite, sanfte Uferübergänge statt abrupter Klippen, damit man an den Ufern noch normal laufen kann.

Die eigentliche Wasserfläche wird nicht als einfache Plane gerendert, sondern als selbst erzeugtes Gitter-Mesh: das Terrain wird in ein feines Raster unterteilt, und nur die Zellen, an denen laut Höhenberechnung tatsächlich Wasser sein soll, bekommen eine Wasser-Fläche. Dadurch folgt das Wasser exakt der Form der Seen und Flüsse.

5. Integration mit dem bestehenden Spieler

Die generierte Karte wird über einen Editor-Menübefehl (**Factory** → **Bake Map Into SampleScene**) fest in die bestehende Spielszene eingebacken - weit entfernt vom Casino platziert, damit sich nichts überschneidet. Danach ist die Karte statischer Szeneninhalte; das Generierungs-Script wird entfernt, es muss beim Spielstart nichts mehr neu berechnet werden.

Ablauf beim Einbacken:

- Ein sicherer Startpunkt auf der Karte wird gesucht (nicht im Wasser, nicht im Gebirge).
- Der Bereich um den Startpunkt wird von Vegetation befreit - ein enger Sicherheitsradius bleibt immer frei (sonst könnte der Spieler in einem Baum feststecken), im äußeren Ring bleiben aber absichtlich ein paar Bäume und Felsen stehen, damit direkt etwas zum Abbauen da ist.
- Der bestehende Spieler (samt Kamera) wird direkt auf diesen Punkt gesetzt - beim Drücken von Play steht man sofort auf der Fabrik-Karte.
- Die ursprüngliche Casino-Position wird als Rückkehrpunkt gespeichert.
- Der Terrain-Layer wird automatisch an die "Ground"-Erkennung des Spieler-Scripts angepasst, damit die Boden-Kollision korrekt funktioniert.

Taste **T** teleportiert jederzeit zurück zum ursprünglichen Casino-Startpunkt und wieder zurück zur Fabrik.

6. Baumodus

Taste **B** schaltet einen Baumodus ein, in dem Maschinen auf einem Raster platziert werden können (Script: **BuildModeController.cs**).

Funktionsweise

- Ein Bodenraster wird dort eingeblendet, wo der Spieler gerade hinschaut (per Raycast aus der Kamera-Mitte auf das Gelände).
- Alle Maschinengrößen sind ganzzahlige Vielfache einer festen Basiszelle (Standard 2×2 Units) - z. B. sind die meisten Maschinen 2×2, ein größerer "Assembler" ist 4×4 (also 2×2 Basiszellen). Nur so lassen sich unterschiedlich große Maschinen lückenlos aneinanderbauen.
- Das Einrasten erfolgt automatisch korrekt: belegt eine Maschine eine ungerade Anzahl Basiszellen, rastet ihre Mitte in der Zellenmitte ein; bei einer geraden Anzahl auf einer Rasterlinien-Kreuzung - nur so stimmen die Ränder exakt mit dem Raster überein.
- Mit den Zifferntasten 1-6 oder dem Mausrad wird die Maschine gewählt (Miner, Smelter, Constructor, Assembler, Conveyor, Storage), mit R lässt sie sich in 90°-Schritten drehen.
- Eine halbtransparente Vorschau zeigt grün (platzierbar) oder rot (blockiert/überlappt) an.
- Linksklick platziert, Rechtsklick unter dem Fadenkreuz entfernt eine Maschine wieder.

Die Maschinen selbst sind aktuell einfache farbige Platzhalter-Quader (kein Spiel-Content für echte Fördermechanik) - das Bau-System (Raster, Platzierung, Kollisionsprüfung) ist aber vollständig funktionsfähig und könnte später um echte Maschinen-Modelle und Funktionalität erweitert werden.

7. Ressourcen-Abbau

Das Script **HarvestInteraction.cs** verbindet Bäume und Erzvorkommen mit dem Inventar: hält man das Fadenkreuz auf einen Baum oder ein Erzvorkommen, erscheint eine Einblendung (z. B. "[H] Eisen abbauen"). Drückt man die Taste **H**, wird eine feste Menge abgebaut und direkt der Hotbar hinzugefügt (siehe Kapitel 8). Ist ein Vorkommen erschöpft, verschwindet es aus der Welt.

Hinweis zur Tastenwahl: Ursprünglich war die Abbau-Taste "F" vorgesehen, das kollidierte jedoch mit der bereits im Casino-System vorhandenen Verkaufsfunktion (ebenfalls "F"). Die Abbau-Taste wurde deshalb auf "H" umgelegt, und ein zusätzlicher Fehler im bestehenden Verkaufs-Script (fehlende Prüfung auf leere Hotbar-Fächer) wurde behoben.

8. Crafting-System und Hotbar-Anbindung

Taste **C** öffnet ein Crafting-Menü am rechten Bildschirmrand (Script: **CraftingMenuController.cs**). Beim Öffnen wird der Mauszeiger automatisch sichtbar und frei nutzbar, gleichzeitig werden Kamera-Steuerung und Hotbar-Tastatur-/Mausrad-Eingaben pausiert, damit man in Ruhe im Menü klicken und scrollen kann, ohne versehentlich die Sicht zu drehen oder die Hotbar-Auswahl zu verändern.

Aufbau des Menüs

- Oben eine Inventar-Übersicht mit allen aktuell vorhandenen Materialien und Mengen.
- Darunter eine scrollbare Liste herstellbarer Gegenstände (aktuell: Eisenplatte, Kupferdraht, Holzbrett, Beton, Zahnrad, Stahlträger).
- Fährt man mit der Maus über ein Rezept, erscheint eine Einblendung mit den genau benötigten Materialien inklusive aktuellem Bestand (z. B. "Iron: 3 / 2").
- Ein Klick craftet den Gegenstand - vorausgesetzt genug Material ist vorhanden (nicht herstellbare Rezepte sind rot markiert).

Verbindung zum bestehenden Inventar

Ein zentraler Entwurfspunkt: Es gibt **kein separates Fabrik-Inventar**. Sowohl abgebaute als auch gecraftete Gegenstände landen direkt in der bereits vorhandenen Casino-Hotbar (unten im Bildschirm). Die Verbindung dazu übernimmt das Script **FactoryItemBridge.cs**: Für jede Materialart wird einmalig ein kleines, unsichtbares Platzhalter-Objekt angelegt, das als eindeutige Kennung in der bestehenden Hotbar-Logik dient. Materialverbrauch beim Craften und neue Gegenstände laufen komplett über die bestehende Hotbar-Funktion des Casino-Systems.

9. Technische Umsetzung und Werkzeuge

Editor-Werkzeuge

Die gesamte Map- und System-Einrichtung läuft über eigene Menübefehle im Unity-Editor (**FactoryMapSetup.cs**, Menüpunkt "Factory"), damit nichts manuell in der Szene zusammengeklickt werden muss:

Befehl	Zweck
Generate New Map (New Scene)	Erzeugt eine komplette neue Testszene mit generierter Karte
Bake Map Into SampleScene	Generiert die Karte fest in die bestehende Spielszene, platziert den Spieler direkt in der Mitte
Regenerate Map In Current Scene	Erzeugt die Karte mit neuem Zufalls-Seed neu (zum Ausprobieren)
Setup Build Mode Only	Rüstet nur den Baumodus nach, ohne die Karte neu zu erzeugen
Setup Crafting Menu Only	Rüstet nur Abbau- und Crafting-System nach, ohne Neu-Generierung

Qualitätssicherung

Jede Änderung wurde vor der Übergabe automatisiert getestet: In einer isolierten Kopie des Projekts wurde Unity im Kommandozeilen-Modus (headless, ohne Grafikausgabe) gestartet, um Kompilierfehler zu erkennen und die Kartengenerierung sowie das Einbacken tatsächlich durchlaufen zu lassen - ohne dabei die eigentliche, im Editor geöffnete Arbeitsversion des Projekts zu beeinflussen oder zu blockieren.

10. Wichtige Code-Stellen erklärt

Dieser Abschnitt erklärt die technisch wichtigsten Ausschnitte aus dem Code im Detail - gedacht als Vorbereitung auf Rückfragen, falls der Code selbst gelesen wird.

10.1 Mehrschichtiges Rauschen (Fractal Noise)

Datei: *MapGenerator.cs*

```
float FractalNoise(float x, float z, float scale,
    int octaves, float persistence, float lacunarity,
    int offX, int offZ)
{
    float amplitude = 1f;
    float frequency = 1f;
    float sum = 0f;
    float maxSum = 0f;

    for (int i = 0; i < octaves; i++)
    {
        float sampleX = (x + offX) / scale * frequency;
        float sampleZ = (z + offZ) / scale * frequency;
        sum += Mathf.PerlinNoise(sampleX, sampleZ) * amplitude;
        maxSum += amplitude;
        amplitude *= persistence;
        frequency *= lacunarity;
    }
    return sum / maxSum; // Ergebnis liegt zwischen 0 und 1
}
```

Mathf.PerlinNoise(x, z) ist eine in Unity eingebaute Funktion, die für jede Koordinate einen glatten, zufällig wirkenden Wert zwischen 0 und 1 liefert - benachbarte Koordinaten ergeben ähnliche, sich weich verändernde Werte (daher "glattes" Rauschen, im Gegensatz zu reinem Zufallsrauschen). Ein einzelner Aufruf allein sieht aber sehr "rund" und gleichförmig aus. Deshalb werden hier mehrere **Oktaven** (Schichten) übereinandergelegt: jede weitere Schicht hat eine höhere Frequenz (`frequency *= lacunarity`, meist $\times 2$ - also feinere Details) und eine geringere Stärke (`amplitude *= persistence`, meist $\times 0,5$ - trägt weniger zum Gesamtbild bei). Am Ende wird durch die Summe aller Amplituden geteilt, damit das Ergebnis wieder zwischen 0 und 1 liegt. **scale** bestimmt, wie weit man "hineinzoomt" - ein großer Wert erzeugt großflächige, sanfte Hügel, ein kleiner Wert viele kleine Unebenheiten. **offX/offZ** verschieben die Abtastposition und sorgen dafür, dass unterschiedliche Aufrufe (z. B. für Höhe, Feuchtigkeit, Baumverteilung) nicht zufällig dieselben Muster erzeugen.

10.2 Domain Warp (Verzerrung der Abtastkoordinaten)

Datei: *MapGenerator.cs*, Methode *GetNormalizedHeight()*

```
float warpX = worldX + (FractalNoise(worldX, worldZ,
    warpNoiseScale, 2, 0.5f, 2f,
    offsetHX + 9001, offsetHZ) - 0.5f) * warpStrength;
float warpZ = worldZ + (FractalNoise(worldX, worldZ,
    warpNoiseScale, 2, 0.5f, 2f,
    offsetHX, offsetHZ + 9001) - 0.5f) * warpStrength;

float raw = FractalNoise(warpX, warpZ, heightNoiseScale,
    heightOctaves, heightPersistence, heightLacunarity,
    offsetHX, offsetHZ);
```

Würde man die Höhe direkt an der Position (worldX, worldZ) abfragen, entstehen bei Perlin-Rauschen typischerweise runde, kraterartige Hügelformen - das Muster ist an der zugrunde liegenden Rechen-Gitterstruktur erkennbar. Der Trick **Domain Warp** berechnet zuerst ein eigenes, unabhängiges Rauschen (mit anderem Offset, deshalb + 9001 als "Kennzeichen" für eine andere Rauschebene) und nutzt dessen Wert, um die eigentliche Abtastposition vor der Höhenberechnung leicht zu verschieben (warpStrength steuert wie stark). Die Höhe wird dann an dieser leicht "verbogenen" Position abgefragt. Das Ergebnis: unregelmäßige, organisch wirkende Hügelketten statt sichtbar rasterartiger Formen.

10.3 Glättung der Höhenkarte (Box-Blur)

Datei: *MapGenerator.cs*, Methode *SmoothHeights()*

```
for (int z = 0; z < res; z++)
for (int x = 0; x < res; x++)
{
    float sum = 0f; int count = 0;
    for (int dz = -1; dz <= 1; dz++)
    for (int dx = -1; dx <= 1; dx++)
    {
        int nx = x + dx, nz = z + dz;
        if (nx < 0 || nx >= res || nz < 0 || nz >= res)
            continue;
        sum += heights[nz, nx];
        count++;
    }
    result[z, x] = sum / count; // Mittelwert der 3x3-Umgebung
}
```

Jeder Punkt der fertigen Höhenkarte wird durch den **Durchschnitt seiner acht Nachbarn plus sich selbst** ersetzt (eine 3×3-Nachbarschaft). Das ist ein klassischer "Box-Blur" aus der Bildbearbeitung, hier auf Höhenwerte statt Pixelfarben angewendet. Die Randprüfung (if (nx < 0 || ...) continue;) verhindert einen Arrayzugriff außerhalb der Karte. Zweck: Durch das Zusammenspiel aus Domain Warp, Ebenen-Überblendung und See-/Fluss-Absenkung konnten an manchen Stellen scharfe, sprunghafte Übergänge zwischen benachbarten Geländepunkten entstehen - genug, damit der Spieler-Charakter (eine Physik-Kapsel) beim Laufen daran hängen bleibt. Der Blur rundet solche Mini-Kanten ab, ohne die große Form der Landschaft merklich zu verändern.

10.4 Raycast + GetComponentInParent (Abbau-Interaktion)

Datei: *HarvestInteraction.cs*

```
Ray ray = cam.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0f));
if (!Physics.Raycast(ray, out RaycastHit hit, harvestDistance))
    return;

ResourceNode node = hit.collider.GetComponentInParent<ResourceNode>();
if (node != null)
{
    if (Input.GetKeyDown(harvestKey))
    {
        int mined = node.Mine(harvestAmountPerPress);
        FactoryItemBridge.Add(node.type.ToString(), mined);
    }
}
```

ViewportPointToRay(0.5, 0.5, 0) erzeugt einen unsichtbaren Strahl aus der Kamera genau durch die **Bildschirmmitte** - also dorthin, wo das Fadenkreuz sitzt. Physics.Raycast prüft, ob dieser Strahl innerhalb von harvestDistance auf einen Collider trifft, und liefert im Erfolgsfall Trefferinformationen (hit) zurück, u. a. den getroffenen Collider. Ein Erzvorkommen besteht aber aus mehreren kleinen Einzelteilen (siehe Kapitel 3) - getroffen wird also z. B. der Collider eines einzelnen "OreBit"-Würfels, nicht das Hauptobjekt. GetComponentInParent<T>() sucht deshalb nicht nur am getroffenen Objekt selbst, sondern läuft die Elternkette (Hierarchie) nach oben, bis es eine Komponente vom gesuchten Typ findet - hier landet man so beim ResourceNode-Script auf dem gemeinsamen Wurzel-Objekt, unabhängig davon, welches Einzelteil genau getroffen wurde.

10.5 Eine gemeinsame Kollisionshülle für mehrteilige Objekte

Datei: *MapGenerator.cs*, Methode *AddBoundingInteractionCollider()*

```
Renderer[] renderers = root.GetComponentsInChildren<Renderer>();
Bounds worldBounds = renderers[0].bounds;
for (int i = 1; i < renderers.Length; i++)
    worldBounds.Encapsulate(renderers[i].bounds);

BoxCollider box = root.AddComponent<BoxCollider>();
box.center = root.transform.InverseTransformPoint(worldBounds.center);
box.size = worldBounds.size;
```

Jedes sichtbare 3D-Objekt hat eine **Bounding Box** (Renderer.bounds) - die kleinstmögliche, achsenausgerichtete Box, die die gesamte sichtbare Form umschließt. Diese Methode holt sich die Bounding Box *aller* Kind-Objekte (GetComponentsInChildren) und vereinigt sie schrittweise zu einer einzigen, alles umfassenden Box (Encapsulate). Anschließend wird ein neuer BoxCollider auf dem Wurzel-Objekt erzeugt, der genau diese Gesamt-Box abdeckt. InverseTransformPoint rechnet die Welt-Koordinate des Mittelpunkts in die *lokale* Koordinate des Wurzel-Objekts um (Collider-Eigenschaften werden relativ zum eigenen Transform gespeichert, nicht in Weltkoordinaten). Ergebnis: ein Raycast trifft immer irgendwo auf diese eine große Box, unabhängig davon, ob er zufällig zwischen zwei der kleinen Einzelteile hindurchgegangen wäre.

10.6 Raster-Einrasten mit gerader/ungerader Zellenanzahl

Datei: BuildModeController.cs

```
float SnapAxis(float p, float size)
{
    int cellsSpan = Mathf.Max(1,
        Mathf.RoundToInt(size / baseGridUnit));
    if (cellsSpan % 2 != 0)
        return (Mathf.Floor(p / baseGridUnit) + 0.5f)
            * baseGridUnit; // ungerade -> Zellenmitte
    return Mathf.Round(p / baseGridUnit) * baseGridUnit;
    // gerade -> Linien-Kreuzung
}
```

Alle Maschinen sind ganzzahlige Vielfache einer Basiszelle (Standard 2×2). `cellsSpan` berechnet, wie viele Basiszellen die aktuelle Maschine entlang dieser Achse belegt (z. B. 1 bei einer 2er-Maschine, 2 bei einer 4er-Maschine). Der Modulo-Operator `% 2` prüft, ob diese Zahl gerade oder ungerade ist - das entscheidet, wo der Mittelpunkt der Maschine einrasten muss, damit ihre *Ränder* exakt auf den Rasterlinien liegen: Bei einer **ungeraden** Anzahl Zellen (z. B. 1) muss die Mitte in einer *Zellenmitte* liegen (`Mathf.Floor(...)` + 0.5f rundet zur nächsten "Halben Stelle"). Bei einer **geraden** Anzahl (z. B. 2) muss die Mitte dagegen genau auf einer *Linienkreuzung* liegen (einfaches `Mathf.Round`). Ohne diese Fallunterscheidung würden größere und kleinere Maschinen beim Aneinanderbauen nicht exakt zusammenpassen.

10.7 Statische Klasse als Brücke zu einem bestehenden System

Datei: FactoryItemBridge.cs

```
public static class FactoryItemBridge
{
    private static GameLogicScript logicScript;
    private static readonly Dictionary<string, GameObject>
        templates = new Dictionary<string, GameObject>();

    public static void Add(string itemName, int amount)
    {
        GameLogicScript script = GetLogicScript();
        script.changeItemInHotbar(
            GetOrCreateTemplate(itemName), amount);
    }
}
```

Eine static class gehört zu keinem GameObject in der Szene und muss nicht wie ein normales Script per `AddComponent` angehängt werden - ihre Methoden sind von überall im Code direkt aufrufbar (`FactoryItemBridge.Add(...)`), ohne vorher eine Instanz zu erzeugen. Das passt hier gut, weil diese Klasse keinen eigenen Zustand in der Welt braucht, sondern nur eine **Vermittler-Funktion** zwischen zwei bestehenden Systemen ist: dem neuen Abbau-/Crafting-Code und dem bereits vorhandenen Hotbar-System (`GameLogicScript`). Das Dictionary `templates` merkt sich für jeden Item-Namen ein einmal erzeugtes "Platzhalter"-Objekt, damit für z. B. "Iron" immer dasselbe Objekt wiederverwendet wird - die bestehende Hotbar-Logik erkennt Items nämlich darüber, ob es sich um *dasselbe* Objekt handelt, nicht über einen separaten Namensvergleich im eigentlichen Sinn eines Item-Systems.

11. Steuerungsübersicht

Taste / Eingabe	Funktion
-----------------	----------

WASD	Bewegen
Maus	Umsehen
Leertaste	Springen
Shift	Sprinten
Strg	Ducken
H	An Baum/Erz: Material abbauen
B	Baumodus an/aus
1-6 / Mausekranz (im Baumodus)	Maschine auswählen
R (im Baumodus)	Maschine drehen
Linksklick (im Baumodus)	Maschine platzieren
Rechtsklick (im Baumodus)	Maschine entfernen
C	Crafting-Menü öffnen/schließen
Mausekranz (im Crafting-Menü)	Rezeptliste scrollen
T	Teleport zwischen Casino und Fabrik-Karte
1-0 / Mausekranz (normal)	Hotbar-Slot wählen
Q	Ausgewählten Hotbar-Gegenstand fallen lassen

12. Dateiübersicht

Alle neuen Scripts liegen im Ordner **Assets/Factory/**, getrennt vom restlichen Casino-Code:

Datei	Aufgabe
MapGenerator.cs	Kernlogik der Kartengenerierung: Höhen, Biome, Texturen, Wasser, Vegetation
ResourceNode.cs	Datenmodell eines Erzvorkommens (Typ, Restmenge, Abbau-Methode)
HarvestableTree.cs	Datenmodell eines Baumes (Holzmenge, Abbau-Methode)
TeleportToFactoryMap.cs	Teleport-Hotkey zwischen Casino und Fabrik-Karte
MachineDefinition.cs	Beschreibung eines Maschinentyps (Größe, Farbe) für den Baumodus
MachineNode.cs	Markiert eine platzierte Maschine in der Welt
BuildModeController.cs	Baumodus: Raster, Vorschau, Platzieren/Entfernen von Maschinen
HarvestInteraction.cs	Verbindet Bäume/Erz mit der Hotbar (Abbau-Taste)
CraftingRecipe.cs	Rezeptliste (herstellbare Gegenstände und Materialkosten)
CraftingMenuController.cs	Das Crafting-Menü selbst (Anzeige, Klick, Tooltip, Scrollen)
FactoryItemBridge.cs	Bindeglied zwischen Factory-System und bestehender Casino-Hotbar
FactoryMapSetup.cs (Editor)	Menübefehle zum Generieren, Einbacken und Nachrüsten der Systeme

Zusätzlich wurden zwei bestehende Scripts minimal erweitert: **CameraRotation.cs** (Kamera-Steuerung pausierbar gemacht, für Crafting-Menü) und **HotbarScript.cs** (Eingaben pausierbar gemacht, verhindert Konflikt mit Crafting-Menü-Scrollen), sowie ein kleiner, bereits vorher vorhandener Fehler in **GameLogicScript.cs** (fehlende Prüfung auf leeres Hotbar-Fach beim Verkaufen) behoben.